

FIRST

- Giả sử văn phạm không chứa luật sinh rỗng
- Ta muốn biết xem **ký hiệu chưa kết thúc A** có thể **dẫn xuất ra chuỗi có ký tự bắt đầu là t** không.
- Định nghĩa tập **FIRST**
 - $\text{FIRST}(A) = \{ t \mid A \Rightarrow^* t\omega \}$
 - Một cách trực quan, **FIRST(A)** là tập các ký hiệu kết thúc mà nó có thể là ký tự bắt đầu của chuỗi được dẫn xuất ra từ A.
- Nếu ta có thể tính được FIRST cho tất cả các ký hiệu chưa kết thúc của văn phạm, ta có thể xây dựng được bảng phân tích LL(1) dễ dàng.

Giải thuật tính tập FIRST

- Khởi tạo, với mỗi ký hiệu chưa kết thúc **A**,
 - **gán** $\text{FIRST}(\mathbf{A}) = \{ \mathbf{t} \mid \mathbf{A} \rightarrow \mathbf{t}\omega \}$
- Lặp lại cho đến khi bảng T không thay đổi nữa:
 - Với mỗi ký hiệu chưa kết thúc **A**,
 - Với mỗi luật sinh có dạng $\mathbf{A} \rightarrow \mathbf{B}\omega$,
 - Gán $\mathbf{FIRST}(\mathbf{A}) = \mathbf{FIRST}(\mathbf{A}) \cup \mathbf{FIRST}(\mathbf{B})$

Ví dụ

STMT → if **EXPR** then **STMT**
 | while **EXPR** do **STMT**
 | **EXPR** ;

EXPR → **TERM** -> id
 | zero? **TERM**
 | not **EXPR**
 | ++ id
 | -- id

TERM → id
 | constant

Ví dụ

STMT → if **EXPR** then **STMT**
 | while **EXPR** do **STMT**
 | **EXPR** ;

EXPR → **TERM** -> id
 | zero? **TERM**
 | not **EXPR**
 | ++ id
 | -- id

TERM → id
 | constant

STMT	EXPR	TERM

Ví dụ

```

STMT →  if EXPR then STMT
          | while EXPR do STMT
          | EXPR ;

```

```

EXPR  →  TERM -> id
          |
          |  zero? TERM
          |  not EXPR
          |  ++ id
          |  -- id

```

TERM $\rightarrow$ id
 | constant

STMT	EXPR	TERM
if while		

Ví dụ

STMT → if **EXPR** then **STMT**
 | while **EXPR** do **STMT**
 | **EXPR** ;

EXPR → **TERM** -> id
 | zero? **TERM**
 | not **EXPR**
 | ++ id
 | -- id

TERM → id
 | constant

STMT	EXPR	TERM
if while	zero? not ++ --	

Ví dụ

STMT → if **EXPR** then **STMT**
 | while **EXPR** do **STMT**
 | **EXPR** ;

EXPR → **TERM** -> id
 | zero? **TERM**
 | not **EXPR**
 | ++ id
 | -- id

TERM → id
 | constant

STMT	EXPR	TERM
if while	zero? not ++ --	id constant

Ví dụ

STMT → if EXPR then STMT
| while EXPR do STMT
| **EXPR ;**

EXPR → TERM → id
| zero? TERM
| not EXPR
| ++ id
| -- id

TERM → id
| constant

STMT	EXPR	TERM
if while	zero? not ++ --	id constant

Ví dụ

STMT → if **EXPR** then **STMT**
| while **EXPR** do **STMT**
| **EXPR** ;

EXPR → **TERM** -> id
| zero? **TERM**
| not **EXPR**
| ++ id
| -- id

TERM → id
| constant

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++		
--		

Ví dụ

STMT → if **EXPR** then **STMT**
 | while **EXPR** do **STMT**
 | **EXPR** ;

EXPR → **TERM** -> id
 | zero? **TERM**
 | not **EXPR**
 | ++ id
 | -- id

TERM → id
 | constant

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++		
--		

Ví dụ

STMT → if EXPR then STMT
| while EXPR do STMT
| EXPR ;

EXPR → **TERM** -> id
| zero? TERM
| not EXPR
| ++ id
| -- id

TERM → id
| constant

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++		
--		

Ví dụ

STMT → if EXPR then STMT
| while EXPR do STMT
| EXPR ;

EXPR → **TERM** -> id
| zero? TERM
| not EXPR
| ++ id
| -- id

TERM → id
| constant

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	

Ví dụ

STMT → if **EXPR** then **STMT**
 | while **EXPR** do **STMT**
 | **EXPR** ;

EXPR → **TERM** -> id
 | zero? **TERM**
 | not **EXPR**
 | ++ id
 | -- id

TERM → id
 | constant

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	

Ví dụ

STMT → if EXPR then STMT
| while EXPR do STMT
| **EXPR ;**

EXPR → TERM → id
| zero? TERM
| not EXPR
| ++ id
| -- id

TERM → id
| constant

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	

Ví dụ

STMT → if **EXPR** then **STMT**
 | while **EXPR** do **STMT**
 | **EXPR** ;

EXPR → **TERM** -> id
 | zero? **TERM**
 | not **EXPR**
 | ++ id
 | -- id

TERM → id
 | constant

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

Ví dụ

STMT → if **EXPR** then **STMT**
 | while **EXPR** do **STMT**
 | **EXPR** ;

EXPR → **TERM** -> id
 | zero? **TERM**
 | not **EXPR**
 | ++ id
 | -- id

TERM → id
 | constant

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

Từ FIRST đến bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** → **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | ++ **id** (7)
 | -- **id** (8)

TERM → **id** (9)
 | **constant** (10)

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT												
EXPR												
TERM												

Từ FIRST đến bảng phân tích LL(1)

STMT $\rightarrow$ **if** **EXPR** **then** **STMT** (1)
 while **EXPR** **do** **STMT** (2)
 EXPR ; (3)

EXPR $\rightarrow$ **TERM** $\rightarrow$ **id** (4)
 zero? **TERM** (5)
 not **EXPR** (6)
 ++ id (7)
 -- id (8)

TERM $\rightarrow$ **id** (9)
 constant (10)

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

	if	then	while	do	zero?	not	++	--	$\rightarrow$	id	const	;
STMT												
EXPR												
TERM												

Từ FIRST đến bảng phân tích LL(1)

STMT $\rightarrow$ **if** **EXPR** **then** **STMT** (1)
 while **EXPR** **do** **STMT** (2)
 EXPR ; (3)

EXPR $\rightarrow$ **TERM** $\rightarrow$ **id** (4)
 zero? **TERM** (5)
 not **EXPR** (6)
 ++ id (7)
 -- id (8)

TERM $\rightarrow$ **id** (9)
 constant (10)

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

	if	then	while	do	zero?	not	++	--	$\rightarrow$	id	const	;
STMT	1											
EXPR												
TERM												

Từ FIRST đến bảng phân tích LL(1)

STMT	→ if EXPR then STMT	(1)
	while EXPR do STMT	(2)
	EXPR ;	(3)
EXPR	→ TERM → id	(4)
	zero? TERM	(5)
	not EXPR	(6)
	++ id	(7)
	-- id	(8)
TERM	→ id	(9)
	constant	(10)

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1											
EXPR												
TERM												

Từ FIRST đến bảng phân tích LL(1)

STMT	→ if EXPR then STMT	(1)
	while EXPR do STMT	(2)
	EXPR ;	(3)
EXPR	→ TERM → id	(4)
	zero? TERM	(5)
	not EXPR	(6)
	++ id	(7)
	-- id	(8)
TERM	→ id	(9)
	constant	(10)

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1		2									
EXPR												
TERM												

Từ FIRST đến bảng phân tích LL(1)

STMT	→ if	EXPR	then	STMT	(1)
	while	EXPR	do	STMT	(2)
		EXPR	;		(3)
EXPR	→	TERM	->	id	(4)
		zero?	TERM		(5)
		not	EXPR		(6)
		++	id		(7)
		--	id		(8)
TERM	→	id			(9)
		constant			(10)

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1		2		3	3	3	3		3	3	
EXPR												
TERM												

Từ FIRST đến bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** → **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM → **id** (9)
 | **constant** (10)

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1		2		3	3	3	3		3	3	
EXPR												
TERM												

Từ FIRST đến bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** → id (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | ++ id (7)
 | -- id (8)

TERM → id (9)
 | constant (10)

STMT	EXPR	TERM
if	zero?	id constant
while	not	
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1		2		3	3	3	3		3	3	
EXPR												
TERM												

Từ FIRST đến bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** → id (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | ++ id (7)
 | -- id (8)

TERM → id (9)
 | constant (10)

STMT	EXPR	TERM
if	zero?	id constant
while	not	
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1		2		3	3	3	3		3	3	
EXPR										4	4	
TERM												

Từ FIRST đến bảng phân tích LL(1)

STMT $\rightarrow$ **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR $\rightarrow$ **TERM** $\rightarrow$ **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM $\rightarrow$ **id** (9)
 | **constant** (10)

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

	if	then	while	do	zero?	not	++	--	$\rightarrow$	id	const	;
STMT	1		2		3	3	3	3		3	3	
EXPR										4	4	
TERM												

Từ FIRST đến bảng phân tích LL(1)

STMT $\rightarrow$ **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR $\rightarrow$ **TERM** $\rightarrow$ **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM $\rightarrow$ **id** (9)
 | **constant** (10)

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

	if	then	while	do	zero?	not	++	--	$\rightarrow$	id	const	;
STMT	1		2		3	3	3	3		3	3	
EXPR					5					4	4	
TERM												

Từ FIRST đến bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** → **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | ++ **id** (7)
 | -- **id** (8)

TERM → **id** (9)
 | **constant** (10)

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1		2		3	3	3	3		3	3	
EXPR					5					4	4	
TERM												

Từ FIRST đến bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** → **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | ++ **id** (7)
 | -- **id** (8)

TERM → **id** (9)
 | **constant** (10)

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1		2		3	3	3	3		3	3	
EXPR					5	6				4	4	
TERM												

Từ FIRST đến bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** → **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | ++ **id** (7)
 | -- **id** (8)

TERM → **id** (9)
 | **constant** (10)

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1		2		3	3	3	3		3	3	
EXPR					5	6				4	4	
TERM												

Từ FIRST đến bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** → **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | ++ **id** (7)
 | -- **id** (8)

TERM → **id** (9)
 | **constant** (10)

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1		2		3	3	3	3		3	3	
EXPR					5	6	7			4	4	
TERM												

Từ FIRST đến bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** → **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM → **id** (9)
 | **constant** (10)

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1		2		3	3	3	3		3	3	
EXPR					5	6	7			4	4	
TERM												

Từ FIRST đến bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** → **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM → **id** (9)
 | **constant** (10)

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1		2		3	3	3	3		3	3	
EXPR					5	6	7	8		4	4	
TERM												

Từ FIRST đến bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** → **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM → **id** (9)
 | **constant** (10)

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1		2		3	3	3	3		3	3	
EXPR					5	6	7	8		4	4	
TERM												

Từ FIRST đến bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** → **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | ++ **id** (7)
 | -- **id** (8)

TERM → **id** (9)
 | **constant** (10)

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1		2		3	3	3	3		3	3	
EXPR					5	6	7	8		4	4	
TERM										9		

Từ FIRST đến bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** → **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM → **id** (9)
 | **constant** (10)

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1		2		3	3	3	3		3	3	
EXPR					5	6	7	8		4	4	
TERM										9		

Từ FIRST đến bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** → **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM → **id** (9)
 | **constant** (10)

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1		2		3	3	3	3		3	3	
EXPR					5	6	7	8		4	4	
TERM										9	10	

Từ FIRST đến bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** → **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM → **id** (9)
 | **constant** (10)

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1		2		3	3	3	3		3	3	
EXPR					5	6	7	8		4	4	
TERM										9	10	

Từ FIRST đến bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** → **id**
 | **zero?** **TERM**
 | **not** **EXPR**
 | ++ **id**
 | -- **id**

TERM → **id**
 | **constant**



STMT	EXPR	TERM
if	zero?	id
	not	constant
	++	
	--	
	id	
	constant	

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1		2		3	3	3	3		3	3	
EXPR					5	6	7	8		4	4	
TERM										9	10	

Bảng phân tích LL(1) không chứa luật sinh rỗng

- Giải thuật
 - Tính FIRST cho tất cả các ký hiệu chưa kết thúc.
 - Với mỗi luật sinh dạng $A \rightarrow t\omega$, gán $T[A, t] = t\omega$.
 - Với mỗi luật sinh dạng $A \rightarrow B\omega$, gán $T[A, t] = B\omega$ với $t \in \text{FIRST}(B)$.

Mở rộng văn phạm

STMT	→	if EXPR then STMT	(1)	id → id;
		while EXPR do STMT	(2)	
		EXPR ;	(3)	while not zero? id do --id;
EXPR	→	TERM -> id	(4)	if not zero? id then
		zero? TERM	(5)	if not zero? id then
		not EXPR	(6)	constant → id;
		++ id	(7)	
		-- id	(8)	
TERM	→	id	(9)	
		constant	(10)	

Mở rộng văn phạm

STMT	→	if EXPR then STMT	(1)	id → id;
		while EXPR do STMT	(2)	
		EXPR ;	(3)	while not zero? id do --id;
EXPR	→	TERM -> id	(4)	if not zero? id then
		zero? TERM	(5)	if not zero? id then
		not EXPR	(6)	constant → id;
		++ id	(7)	
		-- id	(8)	
TERM	→	id	(9)	
		constant	(10)	
BLOCK	→	STMT	(11)	
		{ STMTS }	(12)	
STMTS	→	STMT STMTS	(13)	
		ε	(14)	

Mở rộng văn phạm

STMT	→	if EXPR then BLOCK	(1)	<code>id → id;</code>
		while EXPR do BLOCK	(2)	
		EXPR ;	(3)	<code>while not zero? id do --id;</code>
EXPR	→	TERM -> <code>id</code>	(4)	<code>if not zero? id then</code>
		<code>zero?</code> TERM	(5)	<code>if not zero? id then</code>
		<code>not</code> EXPR	(6)	<code>constant → id;</code>
		<code>++ id</code>	(7)	
		<code>-- id</code>	(8)	
TERM	→	<code>id</code>	(9)	
		<code>constant</code>	(10)	
BLOCK	→	STMT	(11)	
		{ STMTS }	(12)	
STMTS	→	STMT STMTS	(13)	
		ϵ	(14)	

Mở rộng văn phạm

STMT →	if EXPR then BLOCK	(1) <code>id → id;</code>
	while EXPR do BLOCK	(2)
	EXPR ;	(3) <code>while not zero? id do --id;</code>
EXPR →	TERM -> <code>id</code>	(4) <code>if not zero? id then</code>
	zero? TERM	(5) <code>if not zero? id then</code>
	not EXPR	(6) <code>constant → id;</code>
	<code>++ id</code>	(7)
	<code>-- id</code>	(8) <code>if zero? id then</code>
TERM →	<code>id</code>	(9) <code>while zero? id do {</code>
	<code>constant</code>	(10) <code>constant → id;</code>
BLOCK →	STMT	(11) <code>constant → id;</code>
	{ STMTS }	(12) <code>}</code>
STMTS →	STMT STMTS	(13)
	ϵ	(14)

LL(1) cho Văn phạm chứa luật sinh rỗng

- Giải thuật tính FIRST khác với trường hợp văn phạm không có luật sinh rỗng.
 - Phải làm gì nếu ký tự đầu tiên (bên phải) của 1 luật sinh là một ký hiệu chưa kết thúc có thể sinh ra ϵ ?
- Giải thuật xây dựng bảng cũng khác.
 - Làm gì nếu luật sinh sinh ra chuỗi rỗng?

Tính FIRST trong trường hợp luật sinh rỗng

Num	→ Sign Digits
Sign	→ + - ϵ
Digits	→ Digit More
More	→ Digits ϵ
Digit	→ 0 1 2 ... 9

Tính FIRST trong trường hợp luật sinh rỗng

Num → **Sign Digits**
Sign → + | - | ϵ
Digits → **Digit More**
More → **Digits** | ϵ
Digit → 0 | 1 | 2 | ... | 9

Num	Sign	Digit	Digits	More

Tính FIRST trong trường hợp luật sinh rỗng

Num → **Sign Digits**
Sign → + | - | ϵ
Digits → **Digit More**
More → **Digits** | ϵ
Digit → 0 | 1 | 2 | ... | 9

Num	Sign	Digit	Digits	More
	+ -	0 5 1 6 2 7 3 8 4 9		

Tính FIRST trong trường hợp luật sinh rỗng

Num → **Sign Digits**
Sign → + | - | ϵ
Digits → **Digit More**
More → **Digits** | ϵ
Digit → 0 | 1 | 2 | ... | 9

Num	Sign	Digit	Digits	More
	+ -	0 5 1 6 2 7 3 8 4 9		

Tính FIRST trong trường hợp luật sinh rỗng

Num → **Sign Digits**
Sign → + | - | ϵ
Digits → **Digit More**
More → **Digits** | ϵ
Digit → 0 | 1 | 2 | ... | 9

Num	Sign	Digit	Digits	More
+ -	+ -	0 5 1 6 2 7 3 8 4 9		

Tính FIRST trong trường hợp luật sinh rỗng

Num → **Sign Digits**
Sign → **+** | **-** | ϵ
Digits → **Digit More**
More → **Digits** | ϵ
Digit → **0** | **1** | **2** | ... | **9**

Num	Sign	Digit	Digits	More
+ -	+ -	0 5 1 6 2 7 3 8 4 9		

Tính FIRST trong trường hợp luật sinh rỗng

Num → Sign Digits
Sign → + | - | ϵ
Digits → **Digit More**
More → Digits | ϵ
Digit → 0 | 1 | 2 | ... | 9

Num	Sign	Digit	Digits	More
+ -	+ -	0 5 1 6 2 7 3 8 4 9		

Tính FIRST trong trường hợp luật sinh rỗng

Num → Sign Digits
Sign → + | - | ϵ
Digits → **Digit More**
More → Digits | ϵ
Digit → 0 | 1 | 2 | ... | 9

Num	Sign	Digit	Digits	More
+ -	+ -	0 5 1 6 2 7 3 8 4 9	0 5 1 6 2 7 3 8 4 9	

Tính FIRST trong trường hợp luật sinh rỗng

Num → **Sign Digits**
Sign → + | - | ϵ
Digits → **Digit More**
More → **Digits** | ϵ
Digit → 0 | 1 | 2 | ... | 9

Num	Sign	Digit	Digits	More
+ -	+ -	0 5 1 6 2 7 3 8 4 9	0 5 1 6 2 7 3 8 4 9	

Tính FIRST trong trường hợp luật sinh rỗng

Num → Sign Digits
Sign → + | - | ϵ
Digits → Digit More
More → **Digits** | ϵ
Digit → 0 | 1 | 2 | ... | 9

Num	Sign	Digit	Digits	More
+ -	+ -	0 5 1 6 2 7 3 8 4 9	0 5 1 6 2 7 3 8 4 9	

Tính FIRST trong trường hợp luật sinh rỗng

Num → Sign Digits
Sign → + | - | ϵ
Digits → Digit More
More → **Digits** | ϵ
Digit → 0 | 1 | 2 | ... | 9

Num	Sign	Digit	Digits	More
+ -	+ -	0 5 1 6 2 7 3 8 4 9	0 5 1 6 2 7 3 8 4 9	0 5 1 6 2 7 3 8 4 9

Tính FIRST trong trường hợp luật sinh rỗng

Num → **Sign Digits**
Sign → + | - | ϵ
Digits → **Digit More**
More → **Digits** | ϵ
Digit → 0 | 1 | 2 | ... | 9

Num	Sign	Digit	Digits	More
+ -	+ -	0 5	0 5	0 5
		1 6	1 6	1 6
		2 7	2 7	2 7
		3 8	3 8	3 8
		4 9	4 9	4 9

Tính FIRST trong trường hợp luật sinh rỗng

Num → **Sign Digits**
Sign → + | - | ϵ
Digits → **Digit More**
More → **Digits** | ϵ
Digit → 0 | 1 | 2 | ... | 9

Num	Sign	Digit	Digits	More
+ -	+ - ϵ	0 5 1 6 2 7 3 8 4 9	0 5 1 6 2 7 3 8 4 9	0 5 1 6 2 7 3 8 4 9 ϵ

Tính FIRST trong trường hợp luật sinh rỗng

Num → **Sign Digits**
Sign → + | - | ϵ
Digits → **Digit More**
More → **Digits** | ϵ
Digit → 0 | 1 | 2 | ... | 9

Num	Sign	Digit	Digits	More
+ -	+ - ϵ	0 5 1 6 2 7 3 8 4 9	0 5 1 6 2 7 3 8 4 9	0 5 1 6 2 7 3 8 4 9 ϵ

Tính FIRST trong trường hợp luật sinh rỗng

Num → **Sign Digits**
Sign → + | - | ϵ
Digits → **Digit More**
More → **Digits** | ϵ
Digit → 0 | 1 | 2 | ... | 9

Num		Sign	Digit		Digits		More	
+	-	+ - ϵ	0	5	0	5	0	5
0	5		1	6	1	6	1	6
1	6		2	7	2	7	2	7
2	7		3	8	3	8	3	8
3	8		4	9	4	9	4	9
4	9							ϵ

Tính FIRST trong trường hợp luật sinh rỗng

Num → **Sign Digits**
Sign → + | - | ϵ
Digits → **Digit More**
More → **Digits** | ϵ
Digit → 0 | 1 | 2 | ... | 9

Num		Sign	Digit		Digits		More	
+	-	+ - ϵ	0	5	0	5	0	5
0	5		1	6	1	6	1	6
1	6		2	7	2	7	2	7
2	7		3	8	3	8	3	8
3	8		4	9	4	9	4	9
4	9							ϵ

Tính FIRST trong trường hợp luật sinh rỗng

- Khi tính FIRST cho văn phạm chứa luật sinh rỗng, đôi khi ta phải “nhìn xuyên qua” một số ký hiệu chưa kết thúc.
- Lý do: có thể có trường hợp như thế này:
$$\mathbf{A} \Rightarrow \mathbf{Bt} \Rightarrow \mathbf{t}$$
- Vì vậy $\mathbf{t} \in \mathbf{FIRST}(\mathbf{A})$.

Tính FIRST trong trường hợp luật sinh rỗng

- Khởi tạo, với mỗi ký hiệu chưa kết thúc **A**,
 - Gán $\text{FIRST}(\mathbf{A}) = \{ \mathbf{t} \mid \mathbf{A} \rightarrow \mathbf{t}\omega \}$
- Với mỗi ký hiệu chưa kết thúc **A** mà $\mathbf{A} \rightarrow \epsilon$ là **luật sinh**, thêm ϵ vào $\text{FIRST}(\mathbf{A})$.

Tính FIRST trong trường hợp luật sinh rỗng

- Lặp lại cho đến khi không thay đổi nữa:
 - Với mỗi luật sinh $A \rightarrow \alpha$, trong đó α là chuỗi ký hiệu chưa kết thúc mà FIRST của nó chứa rỗng,
 - Gán $\mathbf{FIRST(A) = FIRST(A) \cup \{ \epsilon \}}$.
 - Với mỗi luật sinh $A \rightarrow \alpha t \omega$, trong đó α là chuỗi ký hiệu chưa kết thúc mà FIRST của nó chứa rỗng
 - Gán $\mathbf{FIRST(A) = FIRST(A) \cup \{ t \}}$
 - Với mỗi ký hiệu chưa kết thúc $A \rightarrow \alpha B \omega$, trong đó α là chuỗi ký hiệu chưa kết thúc mà FIRST của nó chứa rỗng
 - Gán $\mathbf{FIRST(A) = FIRST(A) \cup (FIRST(B) - \{ \epsilon \})}$.

Tính FIRST cho chuỗi ký hiệu

- FIRST của chuỗi ω , ký hiệu $\text{FIRST}^*(\omega)$ được định nghĩa như sau:
 - $\text{FIRST}^*(\epsilon) = \{ \epsilon \}$
 - $\text{FIRST}^*(t\omega) = \{ t \}$
 - Nếu $\epsilon \notin \text{FIRST}(A)$:
 - $\text{FIRST}^*(A\omega) = \text{FIRST}(A)$
 - Nếu $\epsilon \in \text{FIRST}(A)$:
 - $\text{FIRST}^*(A\omega) = (\text{FIRST}(A) - \{ \epsilon \}) \cup \text{FIRST}^*(\omega)$

Giải thuật tính FIRST trong trường hợp luật sinh rỗng

- Khởi tạo, với mỗi ký hiệu chưa kết thúc **A**
 - Gán $\text{FIRST}(\mathbf{A}) = \{ \mathbf{t} \mid \mathbf{A} \rightarrow \mathbf{t}\omega \}$
- Với mỗi luật sinh $\mathbf{A} \rightarrow \epsilon$
 - Thêm ϵ vào $\text{FIRST}(\mathbf{A})$.
- Lặp lại cho đến khi không thay đổi nữa:
 - Với mỗi luật sinh $\mathbf{A} \rightarrow \mathbf{a}$,
 - Gán $\text{FIRST}(\mathbf{A}) = \text{FIRST}(\mathbf{A}) \cup \text{FIRST}^*(\mathbf{a})$

Ví dụ

Msg → **Hi End**

Hi → hello | hey | yo

End → world! | ϵ

Ví dụ

Msg → **Hi End**

Hi → hello | heya | yo

End → world! | ϵ

	hello	heyA	yo	world!
Msg				
Hi				
End				

Ví dụ

Msg → **Hi End**

Hi → hello | heya | yo

End → world! | ϵ

Msg	Hi	End

	hello	heya	yo	world!
Msg				
Hi				
End				

Ví dụ

Msg → **Hi End**

Hi → hello | heyá | yó

End → world! | ϵ

Msg	Hi	End
	hello heyá yó	

	hello	heyá	yó	world!
Msg				
Hi				
End				

Ví dụ

Msg → **Hi End**

Hi → hello | heya | yo

End → world! | ϵ

Msg	Hi	End
	hello heya yo	world ϵ

	hello	heya	yo	world!
Msg				
Hi				
End				

Ví dụ

Msg → **Hi End**

Hi → hello | heya | yo

End → world! | ϵ

Msg	Hi	End
	hello heya yo	world ϵ

	hello	heya	yo	world!
Msg				
Hi				
End				

Ví dụ

Msg → **Hi End**

Hi → hello | heya | yo

End → world! | ϵ

Msg	Hi	End
hello heya yo	hello heya yo	world ϵ

	hello	heya	yo	world!
Msg				
Hi				
End				

Ví dụ

Msg → **Hi End**

Hi → hello | heya | yo

End → world! | ϵ

Msg	Hi	End
hello heya yo	hello heya yo	world ϵ

	hello	heya	yo	world!
Msg				
Hi				
End				

Ví dụ

Msg → **Hi End**

Hi → hello | heyA | yo

End → world! | ϵ

Msg	Hi	End
hello heyA yo	hello heyA yo	world ϵ

	hello	heyA	yo	world!
Msg				
Hi				
End				

Ví dụ

Msg → **Hi End**

Hi → hello | heya | yo

End → world! | ϵ

Msg	Hi	End
hello heya yo	hello heya yo	world ϵ

	hello	heya	yo	world!
Msg	Hi End	Hi End	Hi End	
Hi				
End				

Ví dụ

Msg → **Hi End**

Hi → hello | heya | yo

End → world! | ϵ

Msg	Hi	End
hello heya yo	hello heya yo	world ϵ

	hello	heya	yo	world!
Msg	Hi End	Hi End	Hi End	
Hi				
End				

Ví dụ

Msg → **Hi End**

Hi → hello | heya | yo

End → world! | ϵ

Msg	Hi	End
hello heya yo	hello heya yo	world ϵ

	hello	heya	yo	world!
Msg	Hi End	Hi End	Hi End	
Hi	hello	heya	yo	
End				

Ví dụ

Msg → **Hi End**

Hi → hello | heyá | yó

End → world! | ϵ

Msg	Hi	End
hello heyá yó	hello heyá yó	world ϵ

	hello	heyá	yó	world!
Msg	Hi End	Hi End	Hi End	
Hi	hello	heyá	yó	
End				

Ví dụ

Msg → **Hi End**

Hi → hello | heya | yo

End → world! | ϵ

Msg	Hi	End
hello heya yo	hello heya yo	world ϵ

	hello	heya	yo	world!
Msg	Hi End	Hi End	Hi End	
Hi	hello	heya	yo	
End				world!

Ví dụ

Msg → **Hi End**

Hi → hello | heya | yo

End → world! | ϵ

Msg	Hi	End
hello heya yo	hello heya yo	world ϵ

	hello	heya	yo	world!
Msg	Hi End	Hi End	Hi End	
Hi	hello	heya	yo	
End				world!

Ví dụ

	hello	heya	yo	world!
Msg	Hi End	Hi End	Hi End	
Hi	hello	heya	yo	
End				world!

Ví dụ

Msg \$	hello \$
--------	----------

	hello	heya	yo	world!
Msg	Hi End	Hi End	Hi End	
Hi	hello	heya	yo	
End				world!

Ví dụ

Msg \$	hello \$
--------	----------

	hello	heya	yo	world!
Msg	Hi End	Hi End	Hi End	
Hi	hello	heya	yo	
End				world!

Ví dụ

Msg \$	hello \$
Hi End \$	hello \$

	hello	heya	yo	world!
Msg	Hi End	Hi End	Hi End	
Hi	hello	heya	yo	
End				world!

Ví dụ

Msg \$	hello \$
Hi End \$	hello \$

	hello	heya	yo	world!
Msg	Hi End	Hi End	Hi End	
Hi	hello	heya	yo	
End				world!

Ví dụ

Msg \$	hello \$
Hi End \$	hello \$
hello End \$	hello \$

	hello	heya	yo	world!
Msg	Hi End	Hi End	Hi End	
Hi	hello	heya	yo	
End				world!

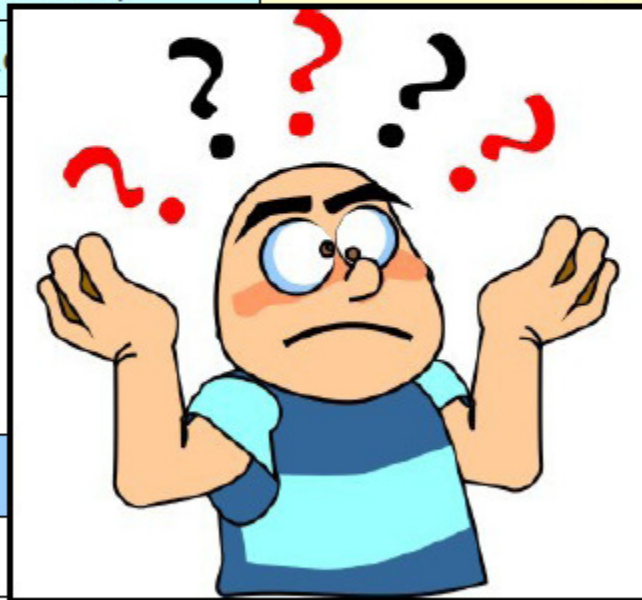
Ví dụ

Msg \$	hello \$
Hi End \$	hello \$
hello End \$	hello \$
End \$	\$

	hello	heya	yo	world!
Msg	Hi End	Hi End	Hi End	
Hi	hello	heya	yo	
End				world!

Ví dụ

Msg \$	hello \$
Hi End \$	hello \$
hello End \$	hello \$
End \$	\$



	hello			world!
Msg	Hi End			
Hi	hello	heya	yo	
End				world!

Ví dụ

- Cần thêm một cột cho \$

Msg → **Hi End**

Hi → hello | heya | yo

End → world! | ϵ

	hello	heya	yo	world!	\$
Msg	Hi End	Hi End	Hi End		
Hi	hello	heya	yo		
End				world!	ϵ

Ví dụ

Msg \$	hello \$
Hi End \$	hello \$
hello End \$	hello \$
End \$	\$

	hello	heya	yo	world!	\$
Msg	Hi End	Hi End	Hi End		
Hi	hello	heya	yo		
End				world!	ϵ

Ví dụ

Msg \$	hello \$
Hi End \$	hello \$
hello End \$	hello \$
End \$	\$
\$	\$

	hello	heya	yo	world!	\$
Msg	Hi End	Hi End	Hi End		
Hi	hello	heya	yo		
End				world!	ϵ

Nhiều luật sinh rỗng

Num → **Sign Digits**
Sign → **+** | **-** | **ε**
Digits → **Digit More**
More → **Digits** | **ε**
Digit → **0** | **1** | ... | **9**

Num		Sign		Digit		Digits		More	
+	-	+	-	0	5	0	5	0	5
0	5	ε		1	6	1	6	1	6
1	6			2	7	2	7	2	7
2	7			3	8	3	8	3	8
3	8			4	9	4	9	4	9
4	9							ε	

	+	-	#	\$
Num	Sign Digits	Sign Digits	Sign Digits	
Sign	+	-	ε	
Digits			Digits More	
More			Digits	ε
Digit			#	

FOLLOW

- Với luật sinh rỗng, ta phải “nhìn xuyên qua” ký hiệu chưa kết thúc để xem sau nó là gì.
- Tập **FOLLOW** biểu diễn tập các ký hiệu kết thúc có thể đi sau một ký hiệu không kết thúc nào đó.
- Chặt chẽ hơn, ta định nghĩa:
 - $\text{FOLLOW}(\mathbf{A}) = \{ \mathbf{t} \mid \mathbf{S} \Rightarrow^* \mathbf{aAtw} \}$ trong đó $\mathbf{S}$ là kí hiệu bắt đầu của văn phạm.